

```

# Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
)

# Models
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

# Gradient Boosting
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# Hyperparameter tuning
from optuna import create_study

# Class imbalance
from imblearn.over_sampling import SMOTE

# Explainable AI
import shap

# Save model
import joblib

```

```
!pip install catboost optuna
```

Collecting catboost

Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl.metadata

Collecting optuna

Downloading optuna-4.9.0-py3-none-any.whl.metadata (15 kB)

Requirement already satisfied: graphviz in /usr/local/lib/python3.12/d

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/

Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/pyt

Requirement already satisfied: pandas<4.0,>=0.24 in /usr/local/lib/pyth

Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-

Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist

Requirement already satisfied: six in /usr/local/lib/python3.12/dist-pa

Collecting alembic>=1.5.0 (from optuna)

```

    Downloading alembic-1.18.5-py3-none-any.whl.metadata (7.2 kB)
Collecting colorlog (from optuna)
  Downloading colorlog-6.11.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from alembic)
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from alembic)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from colorlog)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from colorlog)
Requirement already satisfied: Mako in /usr/lib/python3/dist-packages (from colorlog)
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3.12/dist-packages (from colorlog)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from colorlog)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from colorlog)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from colorlog)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from tqdm)
Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl (97.1 MB)
----- 97.1/97.1 MB 8.2 MB/s
Downloading optuna-4.9.0-py3-none-any.whl (425 kB)
----- 425.6/425.6 kB 29.7 MB/s
Downloading alembic-1.18.5-py3-none-any.whl (264 kB)
----- 264.7/264.7 kB 21.5 MB/s
Downloading colorlog-6.11.0-py3-none-any.whl (12 kB)
Installing collected packages: colorlog, alembic, optuna, catboost
Successfully installed alembic-1.18.5 catboost-1.2.10 colorlog-6.11.0

```

Load Dataset :

```
df = pd.read_csv("/content/sample_data/heart_attack_prediction_indonesia.csv")
```

Data Cleaning :

```
df.shape
```

```
(112019, 28)
```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 112019 entries, 0 to 112018
Data columns (total 28 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   age                                  112019 non-null  int64
 1   gender                              112019 non-null  object
 2   region                              112019 non-null  object
 3   income_level                        112019 non-null  object
 4   hypertension                        112019 non-null  int64
 5   diabetes                            112019 non-null  int64
 6   cholesterol_level                  112019 non-null  int64

```

```

7  obesity 112019 non-null int64
8  waist_circumference 112019 non-null int64
9  family_history 112019 non-null int64
10 smoking_status 112019 non-null object
11 alcohol_consumption 44991 non-null object
12 physical_activity 112019 non-null object
13 dietary_habits 112019 non-null object
14 air_pollution_exposure 112019 non-null object
15 stress_level 112019 non-null object
16 sleep_hours 112018 non-null float64
17 blood_pressure_systolic 112018 non-null float64
18 blood_pressure_diastolic 112018 non-null float64
19 fasting_blood_sugar 112018 non-null float64
20 cholesterol_hdl 112018 non-null float64
21 cholesterol_ldl 112018 non-null float64
22 triglycerides 112018 non-null float64
23 EKG_results 112018 non-null object
24 previous_heart_disease 112018 non-null float64
25 medication_usage 112018 non-null float64
26 participated_in_free_screening 112018 non-null float64
27 heart_attack 112018 non-null float64
dtypes: float64(11), int64(7), object(10)
memory usage: 23.9+ MB

```

```
df.describe()
```

	age	hypertension	diabetes	cholesterol_level	
count	112019.000000	112019.000000	112019.000000	112019.000000	11201
mean	54.547541	0.299565	0.199538	199.546389	
std	11.917980	0.458070	0.399655	39.698740	
min	25.000000	0.000000	0.000000	100.000000	
25%	46.000000	0.000000	0.000000	172.000000	
50%	55.000000	0.000000	0.000000	199.000000	
75%	63.000000	1.000000	0.000000	226.000000	
max	90.000000	1.000000	1.000000	350.000000	

```
df.columns
```

```

Index(['age', 'gender', 'region', 'income_level', 'hypertension',
      'diabetes',
      'cholesterol_level', 'obesity', 'waist_circumference',
      'family_history',
      'smoking_status', 'alcohol_consumption', 'physical_activity',
      'dietary_habits', 'air_pollution_exposure', 'stress_level',
      'sleep_hours', 'blood_pressure_systolic',
      'blood_pressure_diastolic',
      'fasting_blood_sugar', 'cholesterol_hdl', 'cholesterol_ldl',
      'triglycerides', 'EKG_results', 'previous_heart_disease',
      'medication_usage', 'participated_in_free_screening',

```

```
'heart_attack'],  
      dtype='object')
```

```
df["alcohol_consumption"] = df["alcohol_consumption"].fillna("Unknown")
```

```
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

```
print(df["heart_attack"].value_counts())  
print(df["heart_attack"].value_counts(normalize=True) * 100)
```

```
heart_attack  
0.0      66977  
1.0      45041  
Name: count, dtype: int64  
heart_attack  
0.0      59.791284  
1.0      40.208716  
Name: proportion, dtype: float64
```

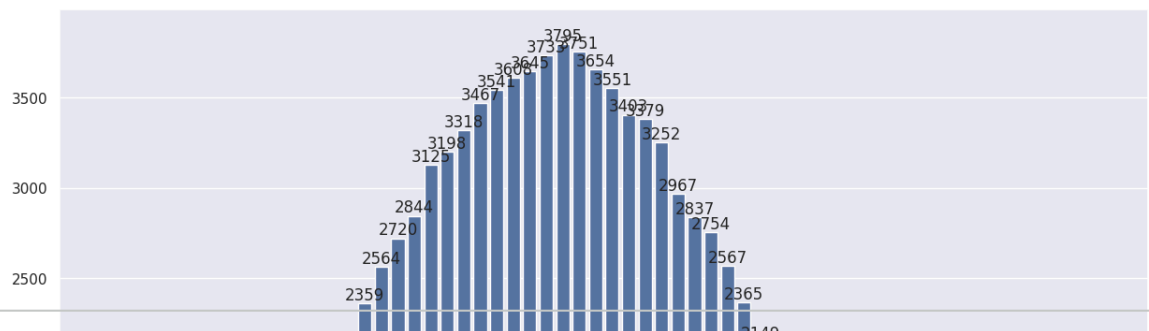
```
for col in df.select_dtypes(include="object").columns:  
    print(f"\n{col}:")  
    print(df[col].unique())
```

```
gender:  
['Male' 'Female']  
  
region:  
['Rural' 'Urban']  
  
income_level:  
['Middle' 'Low' 'High']  
  
smoking_status:  
['Never' 'Past' 'Current']  
  
alcohol_consumption:  
['Unknown' 'Moderate' 'High']  
  
physical_activity:  
['High' 'Moderate' 'Low']  
  
dietary_habits:  
['Unhealthy' 'Healthy']  
  
air_pollution_exposure:  
['Moderate' 'High' 'Low']  
  
stress_level:  
['Moderate' 'High' 'Low' 'Mo']  
  
EKG_results:  
['Normal' 'Abnormal' nan]
```

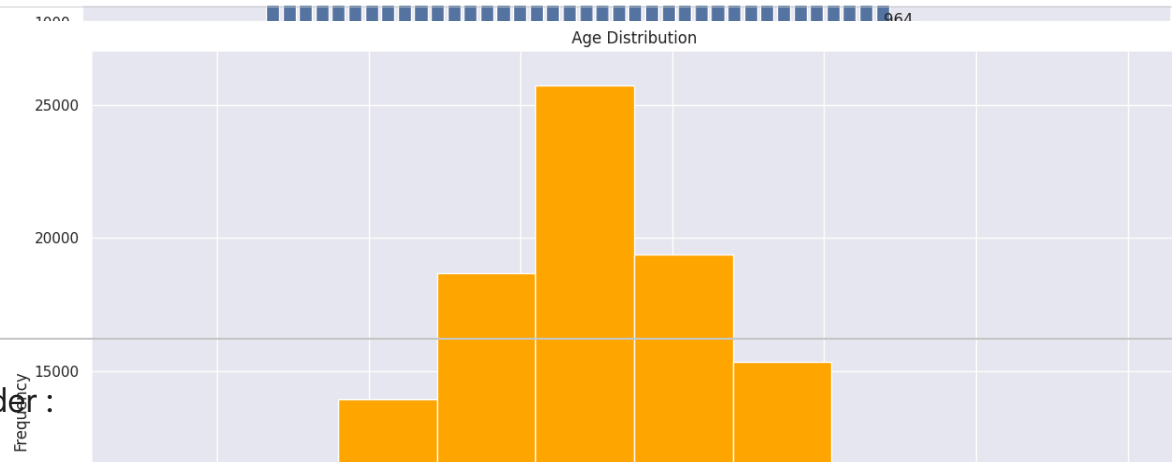
EXPLORATORY DATA ANALYSIS (EDA) :

Age :

```
sns.set(rc = {'figure.figsize':(15,10)})
ax = sns.countplot(data = df,x = "age")
for bars in ax.containers:
    ax.bar_label(bars)
```

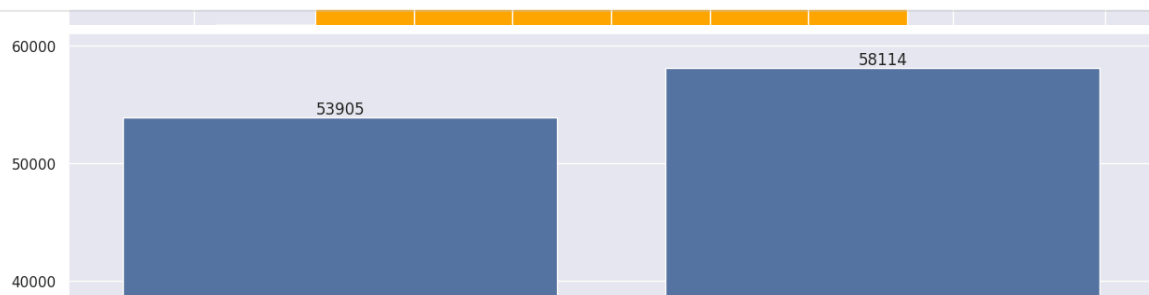


```
plt.hist(df["age"], bins=10,color = "orange")
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



Gender :

```
ax = sns.countplot(data = df, x = "gender",order = [0,1])
for bars in ax.containers:
    ax.bar_label(bars)
```



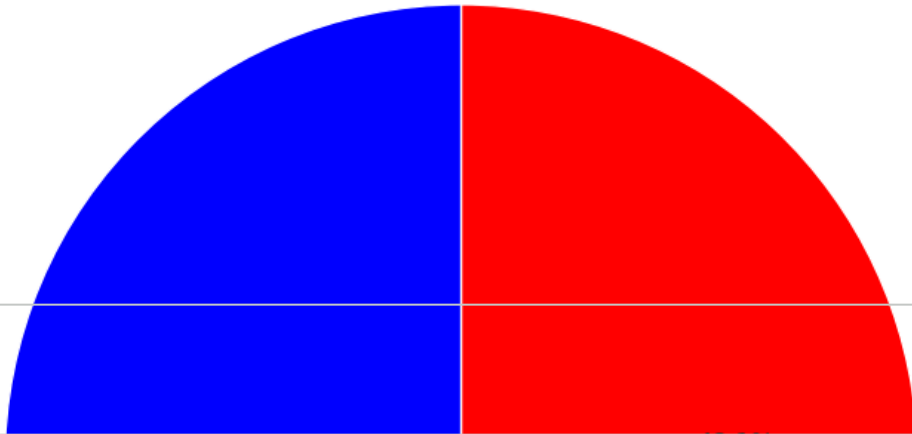
```
df["gender"].value_counts().plot(
kind = "pie",
labels = ["Female","Male"],
autopct = "%1.1f%%",
startangle = 90,
colors =['blue','red'])
```

```

)
plt.title("gender wise")
plt.ylabel("")
plt.show()

```

gender wise



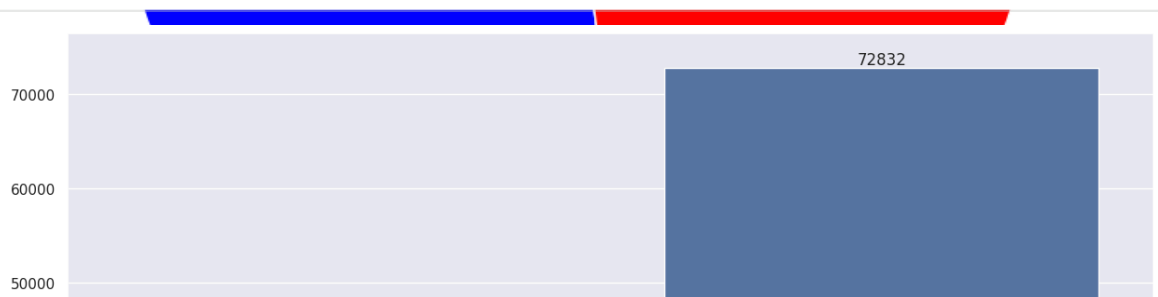
Region :

Male

```

sns.set(rc = {'figure.figsize':(15,10)})
ax = sns.countplot(data = df,x = "region")
for bars in ax.containers:
    ax.bar_label(bars)

```

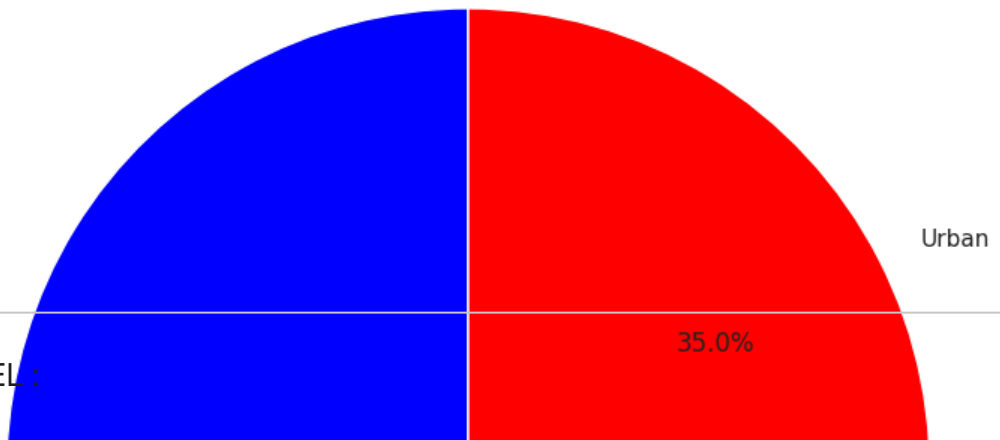


```

df["region"].value_counts().plot(
kind = "pie",
labels= ["Rural","Urban"],
autopct = "%1.1f%%",
startangle = 90,
colors = ['blue','red']
)
plt.title("Region wise")
plt.ylabel("")
plt.show()

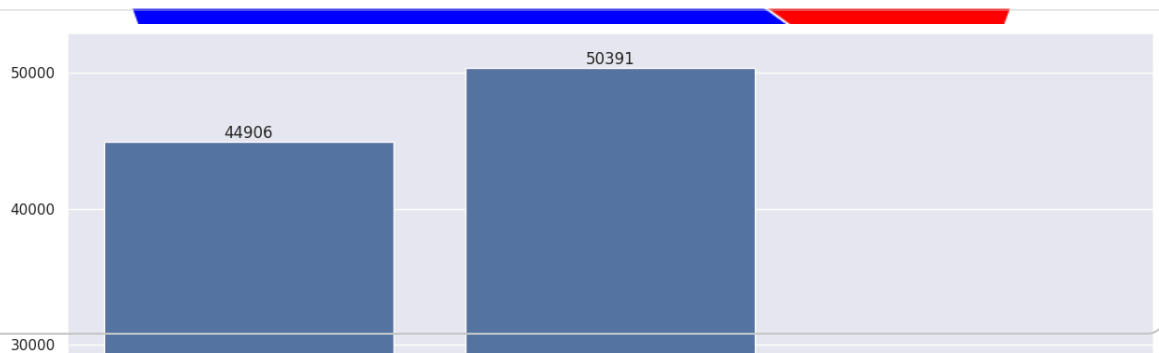
```

Region wise



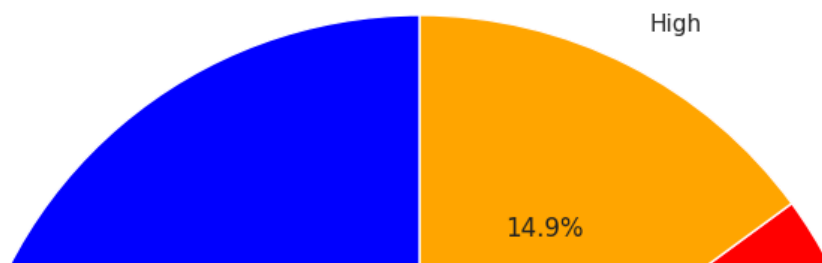
INCOME LEVEL :

```
sns.set(rc = {'figure.figsize':(15,10)})
ax = sns.countplot(data = df,x = "income_level")
for bars in ax.containers:
    ax.bar_label(bars)
```



```
df["income_level"].value_counts().plot(
kind = "pie",
labels= ["Middle","Low","High"],
autopct = "%1.1f%%",
startangle = 90,
colors = ['blue','red','orange']
)
plt.title("Income Wise")
plt.ylabel("")
plt.show()
```

Income Wise

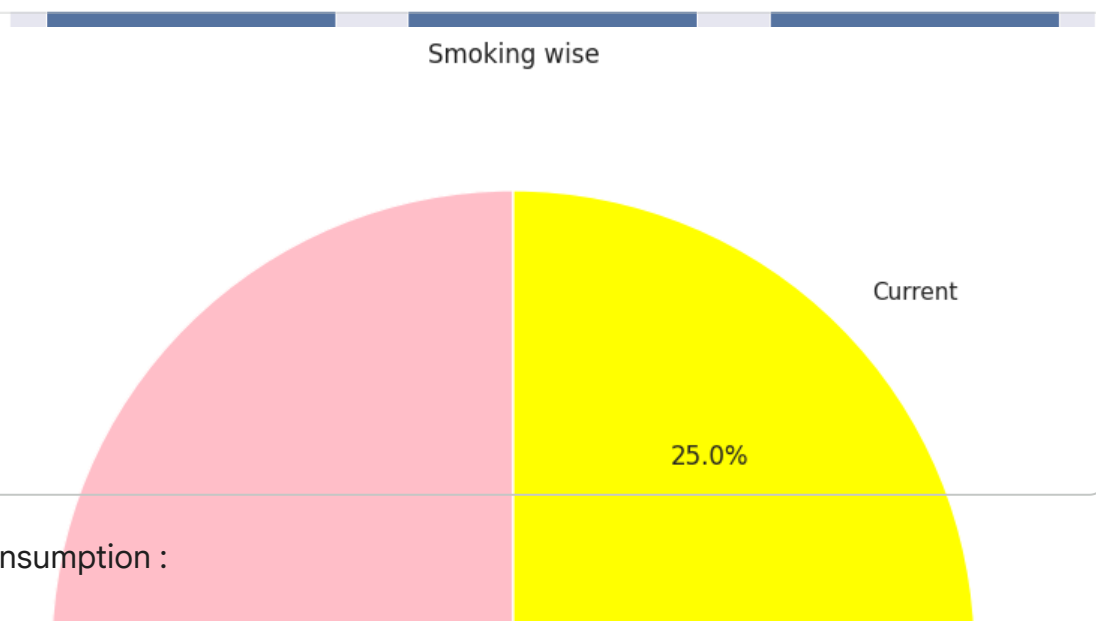


Smoking status :

```
sns.set(rc = {'figure.figsize':(15,10)})
ax = sns.countplot(data = df,x = "smoking_status")
for bars in ax.containers:
    ax.bar_label(bars)
```

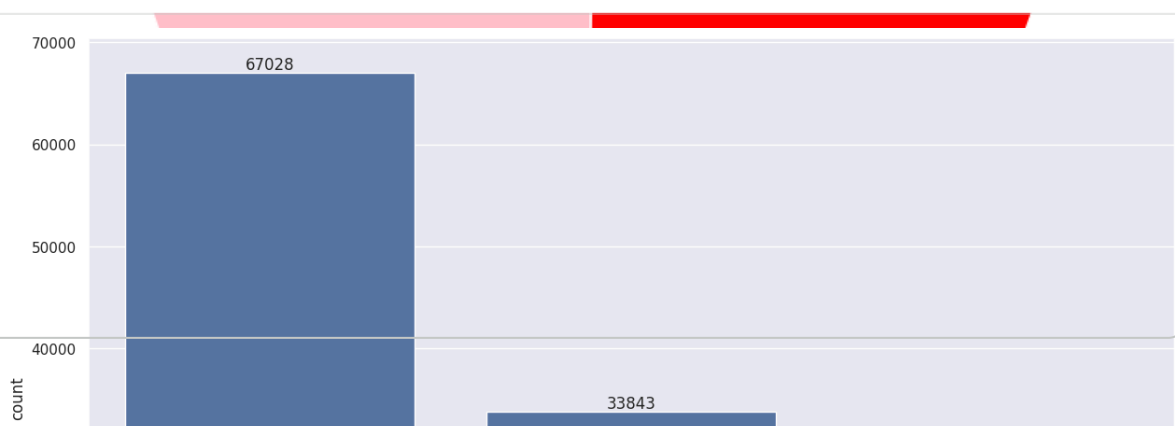


```
df["smoking_status"].value_counts().plot(
kind = "pie",
labels= ["Past","Never","Current"],
autopct = "%1.1f%%",
startangle = 90,
colors = ['pink','red','yellow']
)
plt.title("Smoking wise")
plt.ylabel("")
plt.show()
```



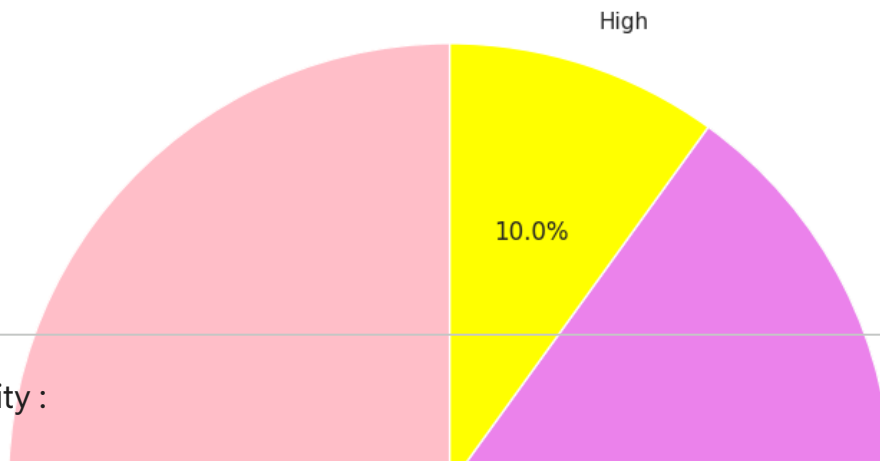
Alcohol consumption :

```
sns.set(rc = {'figure.figsize':(15,10)})
ax = sns.countplot(data = df,x = "alcohol_consumption")
for bars in ax.containers:
    ax.bar_label(bars)
```



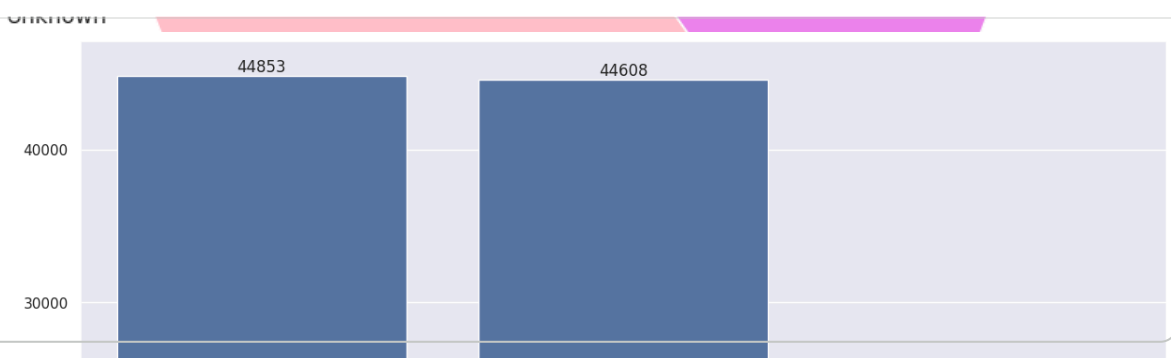
```
df["alcohol_consumption"].value_counts().plot(
kind = "pie",
labels= ["Unknown","Moderate","High"],
autopct = "%1.1f%%",
startangle = 90,
colors = ['pink','Violet','yellow']
)
plt.title("Alcohol Drinking ")
plt.ylabel("")
plt.show()
```

Alcohol Drinking



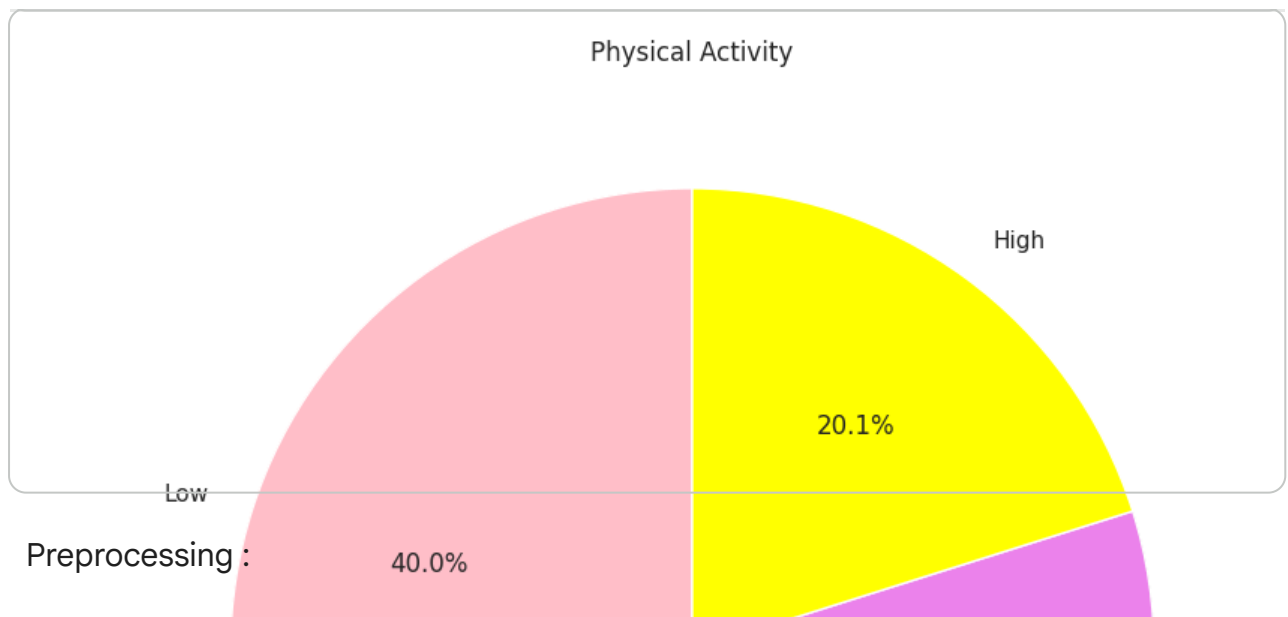
Physical Activity :

```
sns.set(rc = {'figure.figsize':(15,10)})
ax = sns.countplot(data = df,x = "physical_activity")
for bars in ax.containers:
    ax.bar_label(bars)
```



```
df["physical_activity"].value_counts().plot(
kind = "pie",
labels= ["Low","Moderate","High"],
autopct = "%1.1f%%",
startangle = 90,
colors = ['pink','Violet','yellow']
)
plt.title("Physical Activity")
plt.ylabel("")
plt.show()
```





```
# Binary Columns
df["gender"] = df["gender"].map({"Female": 0, "Male": 1})

df["region"] = df["region"].map({"Rural": 0, "Urban": 1})

df["dietary_habits"] = df["dietary_habits"].map({"Unhealthy": 0, "Healthy": 1})

df["EKG_results"] = df["EKG_results"].map({"Normal": 0, "Abnormal": 1})

# Three-Class Columns
df["income_level"] = df["income_level"].map({"Low": 0, "Middle": 1, "High": 2})

df["smoking_status"] = df["smoking_status"].map({"Never": 0, "Past": 1, "Current": 2})

df["alcohol_consumption"] = df["alcohol_consumption"].map({"Unknown": 0, "Moderate": 1, "High": 2})

df["physical_activity"] = df["physical_activity"].map({"Low": 0, "Moderate": 1, "High": 2})

df["air_pollution_exposure"] = df["air_pollution_exposure"].map({"Low": 0, "Moderate": 1, "High": 2})

df["stress_level"] = df["stress_level"].map({"Low": 0, "Moderate": 1, "High": 2})
```

```
for col in df.select_dtypes(include="int").columns:
    print(f"\n{col}:")
    print(df[col].unique())
```

```
age:
[60 53 62 73 52 64 49 61 57 32 34 48 42 58 44 38 72 55 37 56 41 59 47
 77 54 40 31 39 63 46 67 33 50 66 71 25 45 65 84 68 81 43 36 70 35 87
 82 80 30 76 74 29 69 79 78 27 75 28 85 86 83 26 88 89]
```

```
hypertension:
[0 1]
```

```
diabetes:
[1 0]
```

```
cholesterol_level:
[211 208 231 202 232 238 165 186 121 196 190 234 193 125 134 271 185
 132 163 200 191 219 142 180 205 228 265 177 192 176 207 174 225 170
 251 201 159 172 153 258 221 189 214 105 255 149 128 199 131 139 133]
```

```
188 285 252 216 220 212 116 182 250 175 246 215 226 240 210 147 254
243 223 146 241 173 256 244 198 247 187 164 217 218 152 161 303 203
245 151 249 181 178 183 166 184 162 204 156 179 171 158 270 222 136
154 253 167 236 209 266 263 148 242 195 239 235 118 113 229 123 117
233 273 224 272 150 310 298 169 257 268 155 145 213 248 237 160 141
111 275 144 127 264 138 277 305 157 280 119 103 260 135 261 279 140
274 114 115 122 107 100 278 124 283 318 143 306 288 287 120 276 299
129 325 262 106 319 112 293 284 301 281 295 302 296 292 126 289 108
109 267 102 290 300 286 308 312 326 309 291 104 311 330 316 334 294
315 317 341 110 335 323 307 297 324 321 101 314 313 336 320 331 339
327 349 337 328 329 333 332 322 343 348 338 346 340]
```

obesity:

[0 1]

waist_circumference:

```
[ 83 106 112  82  89  81  91  72 115  88  99  80  70  74  77  63 116
 101  84  73 113 132 104 100  90 117 110  92  94 105  97  76  98 108
  93  86  60 124 109  75 148  66  67 111  87  71  96 107 114  69  61
  85  62 103  68  78  58  79  65 130 119 121 123 129 120 135  59  57
  44  50 127 118 131 126  54 143  55 134 128  64  56 133 139  43  52
125  49  46  48 136  45  42 138 141 140 137 173 144 142 151  47 146
  27 149 145  37 155  40  41  33 147 152 150  31 154  20  35 153  32
157  29  38  30 161  36  26 160 163  28]
```

family_history:

[0 1]

blood_pressure_systolic:

```
[113 132 116 136 127 131 128 109 150 142  98 143 146 145 110 138 129
 122 123 117 137 121 130 106 141 118  90  85 100 162 104 126 144 166
 114 135 140 148 107 139 155 151 149 124 115 103 105 119 152 102 147
 134 112  78 153 169 133 120 111 101 159 164  83  88 108  92  94  82
170  97  96 154 163 165  91  89 160  95 156 158  93 168 187  81 167
  74 171  72 174 180  87  86 175 176 172  77  79  84  80 182 181  73
  76 178  75 177 190 183  71  61 185 194 193 184 186 188 199 189  69
 67]
```

blood_pressure_diastolic:

```
[ 62  76  74  65  75  71  97  83  87  98 103  77  54  64  81  49  79
  67  70  89  66  85  78  80  86 105  82  72  68  73  92  90  96  88]
```

```
for col in df.select_dtypes(include="float").columns:
    print(f"\n{col}:")
    print(df[col].unique())
```

gender:

[nan]

region:

[nan]

income_level:

[nan]

smoking_status:

[nan]

```

alcohol_consumption:
[nan]

physical_activity:
[nan]

dietary_habits:
[nan]

air_pollution_exposure:
[nan]

stress_level:
[nan]

sleep_hours:
[5.97060316 5.64381314 6.33619667 ... 8.00867553 6.72772246 8.84349119]

EKG_results:
[nan]

```

```

negative_count = (df["cholesterol_ldl"] < 0).sum()
print("Negative values:", negative_count)

```

```
Negative values: 14
```

```

mask = df["cholesterol_ldl"] < 0

df.loc[mask, "cholesterol_ldl"] = (
    df.loc[mask, "cholesterol_ldl"].abs() + 100
)

```

Train Test split :

```

from sklearn.model_selection import train_test_split

# Features and Target
X = df.drop("heart_attack", axis=1)
y = df["heart_attack"]

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)

```

```

Training set: (126684, 27)
Testing set: (31671, 27)

```

Train Model :

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=500,
    max_depth=20,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)
y_prob = rf_model.predict_proba(X_test)[: , 1]
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
print("ROC AUC  :", roc_auc_score(y_test, y_prob))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy : 0.7243850841463799
Precision: 0.6844745888692744
Recall   : 0.5800787401574803
F1 Score : 0.6279674380940203
ROC AUC  : 0.7937221793562242
```

```
Confusion Matrix:
[[15575  3396]
 [ 5333  7367]]
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.74         0.82         0.78        18971
     1       0.68         0.58         0.63        12700

   accuracy                0.72        31671
  macro avg              0.71         0.70         0.70        31671
 weighted avg              0.72         0.72         0.72        31671
```

XG BOOST

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1,
    eval_metric="logloss"
)

xgb_model.fit(X_train, y_train)
```

```
▼ XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=0.8, device=None, early_stopping_rounds=None,
               enable_categorical=True, eval_metric='logloss',
               feature_types=None, feature_weights=None, gamma=None,
               grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=0.05, max_bin=255,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=6, max_leaves=None,
               min_child_weight=None, missing=nan, monotone_constraints=None,
               multi_strategy=None, n_estimators=500, n_jobs=-1,
               num_parallel_trees=None)
```

```
y_pred = xgb_model.predict(X_test)
y_prob = xgb_model.predict_proba(X_test)[:, 1]
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
print("ROC AUC  :", roc_auc_score(y_test, y_prob))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy : 0.7241956363865998
Precision: 0.6781381975020218
Recall : 0.594251968503937
F1 Score : 0.6334298543791179
ROC AUC : 0.7948086905957166

Confusion Matrix:

```
[[15389  3582]
 [ 5153  7547]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.81	0.78	18971
1	0.68	0.59	0.63	12700
accuracy			0.72	31671
macro avg	0.71	0.70	0.71	31671
weighted avg	0.72	0.72	0.72	31671

Light GBM

```
from lightgbm import LGBMClassifier

lgbm_model = LGBMClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    num_leaves=31,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

lgbm_model.fit(X_train, y_train)
```



```
y_prob = lgbm_model.predict_proba(X_test)[: , 1]
```

```
[LightGBM] [warning] NO further splits with positive gain, best gain.
```

```
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
print("ROC AUC  :", roc_auc_score(y_test, y_prob))
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
n_estimators=500, n_jobs=-1, random_state=42, subsample
```

```
Accuracy : 0.7249850020523507
Precision: 0.6794064748201439
Recall    : 0.5948818897637795
F1 Score  : 0.6343408900083963
ROC AUC   : 0.7953952634709339
```

```
Confusion Matrix:
```

```
[[15406  3565]
 [ 5145  7555]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.75	0.81	0.78	18971
1	0.68	0.59	0.63	12700
accuracy			0.72	31671
macro avg	0.71	0.70	0.71	31671
weighted avg	0.72	0.72	0.72	31671

Cat Boost

```
from catboost import CatBoostClassifier
```

```
cat_model = CatBoostClassifier(
    iterations=500,
    learning_rate=0.05,
    depth=6,
    random_seed=42,
    verbose=0
)
```

```
cat_model.fit(X_train, y_train)
```

```
CatBoostClassifier(depth=6, iterations=500, learning_rate=0.05,
random_seed=42, verbose=0)
```

```
y_pred = cat_model.predict(X_test)
y_prob = cat_model.predict_proba(X_test)[: , 1]
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```

        recall_score,
        f1_score,
        roc_auc_score,
        confusion_matrix,
        classification_report
    )

    print("Accuracy :", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall   :", recall_score(y_test, y_pred))
    print("F1 Score :", f1_score(y_test, y_pred))
    print("ROC AUC  :", roc_auc_score(y_test, y_prob))

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

```

```

Accuracy : 0.7262479871175523
Precision: 0.6834486525855791
Recall   : 0.5911023622047245
F1 Score : 0.6339300793784833
ROC AUC  : 0.7967426411717511

```

```

Confusion Matrix:
[[15494  3477]
 [ 5193  7507]]

```

```

Classification Report:
              precision    recall  f1-score   support

     0       0.75         0.82         0.78        18971
     1       0.68         0.59         0.63        12700

 accuracy          0.73         0.73         0.73        31671
 macro avg         0.72         0.70         0.71        31671
weighted avg         0.72         0.73         0.72        31671

```

Logistic Regression

```
y_train.value_counts()
```

```

              count
heart_attack
0              75883
1              50801

```

```
dtype: int64
```

CatBoost Hyperparameter Tuning

```

from catboost import CatBoostClassifier
from sklearn.model_selection import RandomizedSearchCV

cat_model = CatBoostClassifier(
    random_seed=42,
    verbose=0
)

param_grid = {
    "iterations": [500, 800, 1000],
    "learning_rate": [0.01, 0.03, 0.05, 0.1],
    "depth": [4, 6, 8, 10],
    "l2_leaf_reg": [1, 3, 5, 7],
    "border_count": [32, 64, 128]
}

search = RandomizedSearchCV(
    estimator=cat_model,
    param_distributions=param_grid,
    n_iter=20,
    scoring="roc_auc",
    cv=3,
    random_state=42,
    n_jobs=-1
)

search.fit(X_train, y_train)

print("Best Parameters:")
print(search.best_params_)

print("Best ROC-AUC:")
print(search.best_score_)

```

Best Parameters:

{'learning_rate': 0.01, 'l2_leaf_reg': 1, 'iterations': 800, 'depth': 4

Best ROC-AUC: